

The LangChain Ecosystem

A Reference Guide to Every Package, Module and What It's For

Covers: langchain-core • langchain • langchain-community • langchain-experimental •
langchain-text-splitters • langgraph • langsmith • langserve • langchain-cli • major
partner packages

Compiled as a personal quick-reference. Package internals evolve fast — treat class/function names here as a map of *where to look*, and confirm exact signatures against the installed version's docs before relying on them in code.

Contents

1. The Big Picture — how the packages relate
2. langchain-core — base abstractions & LCEL runtimes
3. langchain — chains, agents, memory (orchestration layer)
4. langchain-community — third-party integrations
5. langchain-text-splitters — chunking utilities
6. langchain-experimental — experimental / higher-risk tools
7. langgraph — stateful multi-step & multi-agent orchestration
8. langsmith — tracing, debugging, evaluation, datasets
9. langserve & langchain-cli — deployment & scaffolding
10. Popular partner (provider) packages

1. The Big Picture

LangChain was split into several packages so that core abstractions, orchestration logic, and third-party integrations can version independently. Roughly:

Module / Submodule	Key Classes & Functions	What it mainly provides
langchain-core	Foundation	Interfaces every other package builds on: messages, prompts, runnables (LCEL), output parsers, base classes for models/tools/retrievers/vector stores. No integrations, minimal dependencies.
langchain	Orchestration	Higher-level building blocks assembled from core primitives: chains, agents, memory, retrieval strategies. Depends on langchain-core; does not itself depend on community.
langchain-community	Integrations	Community-maintained wrappers for hundreds of LLMs, chat models, vector stores, document loaders, tools, and utilities. Optional third-party SDKs installed as needed.
langchain-<provider>	Partner integration	Thin, officially maintained packages for a single provider (openai, anthropic, google-genai, huggingface, pinecone, chroma, mongodb, ...). Preferred over the community versions when available.
langgraph	Agent runtime	Graph-based state machine for building controllable, stateful agents and multi-agent systems; the current recommended way to build custom agent loops.
langsmith	Observability	SDK + platform for tracing runs, debugging chains/agents, managing prompts, and running evaluations. Works independently of which of the above you use.
langserve	Deployment	Wraps a LangChain/LCEL Runnable as a REST API (FastAPI) with auto-generated schema, streaming, and a playground UI.
langchain-cli	Tooling	Command-line scaffolding for new LangChain/LangServe projects and templates.

2. langchain-core

The foundation: interfaces, base classes, and the LCEL runtime. Almost no integrations live here.

```
pip install langchain-core
```

Module / Submodule	Key Classes & Functions	What it mainly provides
langchain_core.messages	HumanMessage, AIMessage, SystemMessage, ToolMessage, FunctionMessage, BaseMessage, trim_messages()	Typed chat message objects passed to/from chat models; message history formatting and trimming.
langchain_core.prompts	PromptTemplate, ChatPromptTemplate, FewShotPromptTemplate, MessagesPlaceholder	Templating for LLM/chat prompts; variable substitution, few-shot example injection.
langchain_core.output_parsers	StrOutputParser, JsonOutputParser, PydanticOutputParser, CommaSeparatedListOutputParser	Parse raw model output into strings, JSON, Pydantic models, lists, etc.
langchain_core.runnables	Runnable, RunnableLambda, RunnableSequence, RunnableParallel, RunnablePassthrough, RunnableBranch, .invoke()/ .batch()/ .stream()/ .ainvoke()	The LCEL (LangChain Expression Language) runtime: composing steps with the ` ` pipe operator, sync/async/streaming/batch execution.
langchain_core.language_models	BaseLLM, BaseChatModel, SimpleChatModel (abstract base classes)	Interfaces every LLM/chat-model integration must implement (generate, stream, bind_tools, with_structured_output).
langchain_core.tools	BaseTool, @tool decorator, StructuredTool, ToolException	Define callable tools with typed args/schemas for agents to invoke.
langchain_core.documents	Document, BaseDocumentTransformer	The standard (page_content, metadata) container used across loaders, splitters, retrievers, vector stores.
langchain_core.vectorstores	VectorStore (abstract base), VectorStoreRetriever	Interface every vector database integration implements: add_texts, similarity_search, as_retriever().
langchain_core.embeddings	Embeddings (abstract base)	Interface for text-embedding providers: embed_documents(), embed_query().
langchain_core.retrievers	BaseRetriever	Interface for anything that returns relevant Documents given a query.
langchain_core.callbacks	BaseCallbackHandler, CallbackManager, StdOutCallbackHandler	Hook into run lifecycle events (on_llm_start, on_tool_end, ...) for logging, streaming, tracing.
langchain_core.example_selectors	SemanticSimilarityExampleSelector, LengthBasedExampleSelector	Pick which few-shot examples to inject into a prompt dynamically.
langchain_core.chat_history	BaseChatMessageHistory, InMemoryChatMessageHistory	Interface + simple implementation for persisting conversation turns.
langchain_core.exceptions / utils	OutputParserException, get_from_env, convert_to_secret_str	Shared error types and small helper utilities used across the ecosystem.

3. langchain

The orchestration layer: pre-built chains, agents, and memory assembled from langchain-core primitives.

```
pip install langchain
```

Module / Submodule	Key Classes & Functions	What it mainly provides
langchain.chains	LLMChain (legacy), RetrievalQA, ConversationalRetrievalChain, create_retrieval_chain, create_stuff_documents_chain, SequentialChain	Ready-made multi-step pipelines (retrieve -> stuff into prompt -> call model -> parse). Many older *Chain classes are being superseded by LCEL compositions.
langchain.agents	AgentExecutor, create_react_agent, create_tool_calling_agent, create_openai_functions_agent, initialize_agent (legacy), AgentType	Build agent loops that decide which tool to call and when, then execute those tools and feed results back to the model.
langchain.memory	ConversationBufferMemory, ConversationBufferWindowMemory, ConversationSummaryMemory, ConversationSummaryBufferMemory, VectorStoreRetrieverMemory	Strategies for persisting/summarizing chat history across turns (largely superseded by langgraph checkpoints + langchain_core.chat_history in newer code).
langchain.retrievers	MultiQueryRetriever, ContextualCompressionRetriever, EnsembleRetriever, ParentDocumentRetriever, SelfQueryRetriever	Higher-level retrieval strategies layered on top of a base VectorStoreRetriever.
langchain.text_splitter	(re-export) RecursiveCharacterTextSplitter, etc.	Convenience re-export of langchain-text-splitters for backward compatibility.
langchain.output_parsers	PydanticOutputParser, StructuredOutputParser, RetryOutputParser, OutputFixingParser	Additional/legacy parsers, including ones that retry or self-correct malformed model output.
langchain.evaluation	load_evaluator, QAEvalChain, CriteriaEvalChain, StringDistanceEvalChain	Programmatic evaluation of chain/agent outputs (correctness, criteria-based grading, string similarity).
langchain.indexes	VectorstoreIndexCreator, index() (sync helper)	Convenience helpers to build/sync a vector index from a set of documents, including de-duplication.
langchain.hub	pull(), push()	Pull/push prompt templates and chains from LangChain Hub (smith.langchain.com/hub).
langchain.globals	set_debug(), set_verbose(), set_llm_cache()	Global toggles for verbose/debug logging and response caching across all chains.

4. langchain-community

The integration warehouse: hundreds of community-maintained wrappers around third-party services.

```
pip install langchain-community
```

Module / Submodule	Key Classes & Functions	What it mainly provides
langchain_community.llms	Ollama, HuggingFaceHub, Cohere, Replicate, LlamaCpp, GPT4All, Bedrock, ...	Wrappers for text-completion style LLM providers not covered by a dedicated partner package.
langchain_community.chat_models	ChatOllama, ChatCohere, ChatBaichuan, ChatZhipuAI, ChatLiteLLM, ...	Chat-style model wrappers for providers without a standalone partner package.
langchain_community.document_loaders	PyPDFLoader, WebBaseLoader, CSVLoader, UnstructuredFileLoader, NotionDBLoader, GitLoader, YoutubeLoader, S3FileLoader, ... (150+)	Load raw content from files, URLs, databases, and SaaS apps into `Document` objects.
langchain_community.vectorstores	FAISS, Chroma, Milvus, Weaviate, Qdrant, Redis, ElasticVectorSearch, SupabaseVectorStore, ...	VectorStore implementations for dozens of vector databases (several have since split into their own partner packages, e.g. langchain-chroma).
langchain_community.embeddings	HuggingFaceEmbeddings, OllamaEmbeddings, CohereEmbeddings, SentenceTransformerEmbeddings, ...	Text embedding providers not covered by a dedicated partner package.
langchain_community.tools	DuckDuckGoSearchRun, WikipediaQueryRun, ShellTool, RequestsGetTool, YouTubeSearchTool, SQLDatabaseToolkit, ...	Ready-made tools agents can call: web search, shell access, HTTP requests, database querying, and more.
langchain_community.utilities	SQLDatabase, GoogleSearchAPIWrapper, WikipediaAPIWrapper, PythonREPL	Thin API/service wrappers that power the `tools` above and can also be used standalone.
langchain_community.retrievers	BM25Retriever, TFIDFRetriever, AzureAISearchRetriever, KNNRetriever	Additional retriever implementations, including classic sparse/keyword methods (BM25, TF-IDF).
langchain_community.graphs	Neo4jGraph, NetworkxEntityGraph	Interfaces for graph databases used in GraphRAG-style pipelines.
langchain_community.callbacks	WandbCallbackHandler, LangFuseCallbackHandler, ArgillaCallbackHandler	Integrations that stream run/trace data into third-party observability tools.
langchain_community.document_transformers	Html2TextTransformer, EmbeddingsRedundantFilter	Post-process/clean documents after loading and before splitting/embedding.

5. langchain-text-splitters

Chunking utilities for breaking long documents into model- and retrieval-friendly pieces.

```
pip install langchain-text-splitters
```

Module / Submodule	Key Classes & Functions	What it mainly provides
character	CharacterTextSplitter	Naive splitting on a single separator (e.g. \n\n) with a max chunk size and overlap.
character (recursive)	RecursiveCharacterTextSplitter	The default general-purpose splitter: tries a list of separators in order (paragraph, sentence, word) to keep chunks semantically coherent.
token	TokenTextSplitter, SentenceTransformersTokenTextSplitter	Splits by token count (tiktoken or model-specific tokenizer) instead of raw characters.
markdown	MarkdownHeaderTextSplitter, MarkdownTextSplitter	Splits Markdown while preserving header hierarchy as chunk metadata.
html	HTMLHeaderTextSplitter, HTMLSectionSplitter	Splits HTML documents along header/section tags.
code	RecursiveCharacterTextSplitter.from_language()	Language-aware splitting for source code (Python, JS, Java, etc.) that respects function/class boundaries.
json	RecursiveJsonSplitter	Splits nested JSON structures into size-bounded chunks while preserving structure.
base	TextSplitter (abstract base), Language (enum)	Shared base class and language enum used by the splitters above.

6. langchain-experimental

Higher-risk / research-stage features not stable or safe enough for the main package.

```
pip install langchain-experimental
```

Module / Submodule	Key Classes & Functions	What it mainly provides
<code>langchain_experimental.autonomous_agents</code>	AutoGPT, BabyAGI	Early autonomous, self-directed task-planning agent implementations.
<code>langchain_experimental.plan_and_execute</code>	PlanAndExecute, load_agent_executor, load_chat_planner	Separates planning (produce a multi-step plan) from execution (run each step), rather than one ReAct loop.
<code>langchain_experimental.sql</code>	SQLDatabaseChain	Natural-language-to-SQL querying against a database (experimental due to injection/execution risk).
<code>langchain_experimental.tools</code>	PythonREPLTool	Executes arbitrary Python code generated by the model — powerful but requires a sandboxed environment.
<code>langchain_experimental.cpal</code>	CPALChain (Causal Program-Aided Language)	Structured reasoning chain that models causal relationships explicitly instead of free-form CoT.
<code>langchain_experimental.graph_transformers</code>	LLMGraphTransformer	Uses an LLM to extract a knowledge graph (nodes/edges) from unstructured text.

Note: Tools in this package (esp. PythonREPLTool, SQLDatabaseChain) execute model-generated code/queries. Only use them in sandboxed or tightly access-controlled environments.

7. langgraph

Graph-based runtime for building controllable, stateful agents and multi-agent systems.

```
pip install langgraph
```

Module / Submodule	Key Classes & Functions	What it mainly provides
langgraph.graph	StateGraph, MessageGraph, START, END, add_node(), add_edge(), add_conditional_edges()	Define an agent's control flow as a graph of nodes (steps) and edges (transitions), including conditional branching and loops.
langgraph.prebuilt	create_react_agent, ToolNode, tools_condition	Ready-made ReAct-style tool-calling agent so common cases need no custom graph.
langgraph.checkpoint	MemorySaver, SqliteSaver, PostgresSaver	Persist graph state between steps/runs — enables pause/resume, time-travel debugging, and durable long-running agents.
langgraph.types	Command, Send, interrupt()	Control-flow primitives for dynamic routing and human-in-the-loop interrupts within a running graph.
langgraph.graph.message	add_messages (reducer)	State-update reducer that appends new chat messages to a running conversation state instead of overwriting it.
langgraph-supervisor / swarm (related pkgs)	create_supervisor, create_swarm	Higher-level multi-agent orchestration patterns (a supervisor delegating to sub-agents, or peer agents handing off to each other).

langgraph is now the recommended way to build custom agents/loops going forward, in place of langchain.agents.AgentExecutor, when you need explicit control over state, branching, retries, or multi-agent handoffs.

8. langsmith

Observability SDK + platform: tracing, debugging, prompt management, and evaluation.

```
pip install langsmith
```

Module / Submodule	Key Classes & Functions	What it mainly provides
langsmith (client)	Client, @traceable decorator	Send trace data (inputs/outputs/latency/errors) for any function or LCEL run to the LangSmith platform for inspection.
langsmith.evaluation	evaluate(), aevaluate(), LangChainStringEvaluator	Run a dataset of examples through your chain/agent and score outputs with built-in or custom evaluators.
langsmith.schemas	Run, Example, Dataset	Typed objects representing a traced run, a labeled example, and a dataset of examples.
langsmith.run_helpers	trace(), get_current_run_tree()	Context managers / helpers for manually grouping and annotating traced spans.
hub (via langchain.hub or langsmith)	pull(), push()	Version-controlled storage and retrieval of prompt templates shared across a team.

9. langserve & langchain-cli

Turning a chain into a deployable API, and scaffolding new projects.

```
pip install langserve langchain-cli
```

Module / Submodule	Key Classes & Functions	What it mainly provides
langserve	<code>add_routes(app, runnable)</code>	Mounts any LCEL Runnable onto a FastAPI app as REST endpoints (<code>/invoke</code> , <code>/batch</code> , <code>/stream</code>) with auto-generated OpenAPI schema.
langserve	<code>RemoteRunnable</code>	Client-side stub that lets another process call a deployed LangServe endpoint as if it were a local Runnable.
langserve (playground)	<code>/playground</code> UI (auto-mounted)	Auto-generated web UI for manually testing a deployed chain's inputs/outputs and streaming.
langchain-cli	<code>langchain app new</code> , <code>langchain app add</code> , <code>langchain serve</code>	Scaffolds a new LangServe project and pulls in pre-built LangChain "templates" (reference RAG/agent implementations).

10. Popular Partner (Provider) Packages

Officially maintained, single-provider packages — thinner and more current than the community equivalents.

```
pip install langchain-openai / langchain-anthropic / ...
```

Module / Submodule	Key Classes & Functions	What it mainly provides
langchain-openai	ChatOpenAI, OpenAIEmbeddings, AzureChatOpenAI	Chat, completion, and embedding models from OpenAI / Azure OpenAI, incl. function/tool-calling and structured output.
langchain-anthropic	ChatAnthropic	Claude chat models: tool calling, structured output, streaming, prompt caching, extended thinking.
langchain-google-genai / langchain-google-vertexai	ChatGoogleGenerativeAI, VertexAI, VertexAIEmbeddings	Gemini models via the Google AI Studio API or Vertex AI, plus Vertex-hosted embeddings.
langchain-huggingface	HuggingFaceEndpoint, ChatHuggingFace, HuggingFaceEmbeddings	Run open models via the HF Inference API/Endpoints or locally through <code>transformers`/sentence-transformers`</code> .
langchain-pinecone	PineconeVectorStore	First-party VectorStore integration for the Pinecone managed vector database.
langchain-chroma	Chroma	First-party VectorStore integration for the Chroma embedded/local vector database.
langchain-mongodb	MongoDBAtlasVectorSearch	Vector search integration for MongoDB Atlas.
langchain-mistralai	ChatMistralAI, MistralAIEmbeddings	Chat and embedding models from Mistral AI.
langchain-groq	ChatGroq	Chat models served on Groq's low-latency inference hardware.
langchain-ollama	ChatOllama, OllamaEmbeddings	First-party integration for locally-run models served by Ollama.
langchain-aws	ChatBedrock, BedrockEmbeddings	Chat and embedding models via Amazon Bedrock.

Rule of thumb: if a partner package exists for the provider you're using, prefer it over the equivalent class in langchain-community — partner packages track that provider's API more closely and are updated more frequently.